

Hilos POSIX como librería de usuario: estudio e implementación del trabajo de Mueller (1993)

Dario Ricardo Nuñez Villacorta

Profesores: Jorge Gonzalez, Mariana Capuñay Asistente: Sofía Ku

Lima, septiembre de 2026

Resumen. Este informe estudia el artículo *A Library Implementation of POSIX Threads under UNIX* de Frank Mueller (Winter USENIX 1993), que defiende que los hilos POSIX pueden implementarse enteramente como una librería de usuario sin soporte del kernel. Primero explico los mecanismos que propone: el kernel de la librería con su kernel flag y dispatcher flag, el diferimiento de señales, las fake calls, los protocolos de herencia y techo de prioridad y el perverted scheduling. Luego presento mini-pthreads, una librería de hilos de usuario en C que reproduce esas ideas sobre Linux usando `ucontext`, `sigaction` y `setitimer`. Cuatro pruebas validan el ciclo de vida de los hilos, la sincronización, la herencia de prioridad en el escenario de la Figura 5 del paper y las fake calls con señales reales. Cierro con los hallazgos al comparar ambas implementaciones y las diferencias que quedaron respecto del original.

1 Introducción

Cuando uno programa con hilos hoy en día casi nunca se pregunta quién los está planificando. En Linux, `pthread.create` termina en una llamada al sistema y el núcleo del sistema operativo se encarga del resto. Pero esa no es la única forma posible de tener hilos, y a inicios de los noventa ni siquiera era la más común: muchos sistemas UNIX no tenían soporte de hilos en el kernel, y si uno quería hilos tenía que construirlos por encima, en el espacio de usuario.

El artículo que me tocó, *A Library Implementation of POSIX Threads under UNIX* de Frank Mueller [1], trata justamente de eso. Mueller implementa la interfaz de hilos que estaba definiendo el borrador POSIX 1003.4a (lo que después sería *Pthreads*) como una librería de C que corre enteramente en modo usuario sobre SunOS 4 en máquinas SPARC, sin tocar el kernel de UNIX. Su tesis es que esto no solo es posible, sino que puede ser tan rápido o más que las implementaciones comerciales que sí tenían ayuda del kernel.

Este informe tiene dos partes. En la primera explico el artículo: qué problema ataca, qué decisiones de diseño toma y por qué, y qué resultados reporta. En la segunda describo mi propia implementación, una librería pequeña de hilos de usuario en C que llamé *mini-pthreads*, en la que reproduzco los mecanismos centrales del paper sobre Linux actual. Cierro con las pruebas que ejecuté, lo que encontré al analizarlas y las diferencias que quedaron entre mi versión y la de Mueller.

No intenté replicar el sistema completo (el original tiene soporte para Ada, cancelación, datos por hilo y bastante más). Mi objetivo fue quedarme con las ideas que sostienen el argumento del paper y hacerlas visibles con programas cortos que se puedan ejecutar y

leer.

2 Contexto: el artículo de Mueller

2.1 El problema

Un hilo es un flujo de control independiente dentro de un proceso. Comparte con los demás hilos del proceso la memoria global, los archivos abiertos y el resto de recursos, pero tiene su propia pila, sus propias variables locales y su propio contador de programa. Se les llama *livianos* porque su contexto es más pequeño que el de un proceso, lo que en principio hace más barato cambiar de uno a otro.

El estándar POSIX de hilos que se estaba votando en 1992 exigía bastante más que crear hilos y alternar entre ellos. Pedía planificación por prioridades con apropiación (*preemption*), manejo de señales por hilo, mutex, variables de condición, datos específicos de cada hilo y cancelación. Mueller distingue tres maneras de cumplir con todo eso:

- **Implementación en el kernel:** toda la funcionalidad vive dentro del sistema operativo. Es más simple de controlar, sobre todo en lo que respecta a señales, pero cada operación paga el costo de entrar y salir del kernel.
- **Implementación de librería:** todo vive en el programa del usuario y se enlaza como cualquier otra librería. Se evita entrar al kernel, pero ahora hay dos planificadores (el de procesos, en el kernel, y el de hilos, en la librería) y el manejo de señales se complica mucho.

- **Una mezcla** de las dos anteriores, que era lo que hacían SunOS y LynxOS.

El artículo defiende la segunda opción. El argumento cuantitativo es sencillo: si entrar al “kernel” de la librería es mucho más barato que entrar al kernel de UNIX, y la mayoría de operaciones de hilos se pueden resolver sin pedirle nada al sistema operativo, la librería debería ganar.

2.2 Objetivos de diseño

Mueller enumera seis objetivos que guían todo el trabajo. Los resumo porque varios de ellos reaparecen directamente en mi implementación:

1. **Apropiabilidad.** Para tener round-robin, prioridades y señales asíncronas, el diseño tiene que permitir quitarle el procesador a un hilo en cualquier momento, no solo cuando este cede voluntariamente.
2. **Cambios de contexto rápidos.** El cambio de contexto es la única forma de pasar el control de un hilo a otro, así que su costo lo pagan todas las operaciones.
3. **Secciones críticas pequeñas,** y baratas de abrir y cerrar.
4. **Sin crecimiento ilimitado de la pila.** Si llega una señal mientras se atiende otra, y luego otra más, cada manejador se apila sobre el anterior. Hay que evitar que eso crezca sin control.
5. **Pocas llamadas al sistema operativo,** especialmente en los caminos que se ejecutan muchas veces, como el cambio de contexto y la atención de señales.
6. **Interfaz independiente del lenguaje,** pensando en que la librería se usaría para implementar las tareas de Ada.

2.3 El kernel de la librería

La idea más importante del paper, y la que más me costó entender al principio, es que la librería tiene su propio “kernel”. No es un kernel en el sentido de sistema operativo: es simplemente una zona de código de la librería que se ejecuta en exclusión mutua respecto de los demás hilos. Todas las estructuras internas (colas de listos, listas de espera de los mutex, bloques de control de los hilos) solo se modifican dentro de esa zona.

El problema que resuelve es el siguiente. En una librería de usuario la apropiación llega por una señal de UNIX, típicamente SIGALRM generada por un temporizador. Esa señal puede llegar en cualquier instrucción, incluida la mitad de una operación que está

insertando un hilo en la cola de listos. Si el manejador de la señal decide cambiar de hilo en ese momento, el dispatcher va a trabajar sobre una cola a medio modificar.

Mueller lo resuelve con un *monitor monolítico*: una sola sección crítica para toda la librería, controlada por dos variables.

- El **kernel flag** indica que se está ejecutando código dentro del kernel de la librería. Entrar consiste simplemente en ponerlo en 1.
- El **dispatcher flag** indica que, al salir del kernel, hay que llamar al dispatcher porque algo cambió: se creó un hilo de mayor prioridad, alguien se bloqueó, o llegó una señal.

Al salir del kernel, si el dispatcher flag está apagado basta con bajar el kernel flag. Si está encendido, se invoca al dispatcher, que puede terminar en un cambio de contexto. Mueller elige un único monitor para toda la librería (en lugar de un candado por estructura) porque su implementación es para uniprosesor, y ahí el grano grueso no cuesta concurrencia y sí ahorra operaciones.

2.4 Señales y dispatcher

Durante la inicialización la librería instala un *manejador universal* para todas las señales de UNIX que se pueden capturar. Cuando llega una señal hay dos casos:

- Si el kernel flag está apagado, el manejador entra al kernel, dirige la señal al hilo que corresponde y llama al dispatcher. Puede que el control no vuelva al hilo interrumpido si la señal despertó a uno de mayor prioridad.
- Si el kernel flag está encendido, la señal **se registra y se difiere**. El manejador retorna de inmediato al punto de interrupción y la señal se atiende recién cuando el dispatcher se ejecute al salir del kernel.

El dispatcher, antes de transferir el control al nuevo hilo, revisa si llegaron señales mientras estaba en el kernel. Si llegaron, las atiende y vuelve a intentar el despacho, porque atender una señal puede cambiar cuál es el hilo que debe correr. Hay un detalle adicional que el paper subraya: cuando el hilo elegido fue interrumpido por una señal, en su pila todavía está pendiente el manejador universal. Por eso el dispatcher enmascara las señales antes de conmutar a él; de lo contrario podría llegar otra señal y apilarse otro manejador encima antes de que el primero retorne, que es exactamente el crecimiento ilimitado de la pila del objetivo 4.

Para decidir a qué hilo va una señal, el paper usa un orden de precedencia: primero el hilo al que estaba

dirigida explícitamente, luego el que la causó (señales síncronas), luego el que armó el temporizador, luego el que pidió la E/S, luego cualquier hilo que la tenga desenmascarada y, si no hay ninguno, se deja pendiente a nivel de proceso.

2.5 Fake calls

Si un hilo registró un manejador propio para una señal, el estándar exige que ese manejador corra con la prioridad de ese hilo. Eso impide llamarlo directamente desde el manejador universal, porque en ese momento puede estar corriendo otro hilo, de otra prioridad.

La solución de Mueller es la *fake call*. En lugar de llamar al manejador, la librería empuja sobre la pila del hilo receptor un marco que simula una llamada a función (un *wrapper*) y ajusta su contador de programa y su puntero de pila. Cuando ese hilo sea despachado, lo primero que ejecutará será el wrapper, que guarda el **errno** del hilo, llama al manejador de usuario, restaura **errno** y la máscara de señales, y finalmente devuelve el control al punto exacto donde el hilo había sido interrumpido. El efecto es que el manejador se ejecuta “dentro” del hilo correcto y a su prioridad, aunque la señal haya llegado en cualquier momento.

2.6 Sincronización e inversión de prioridades

La librería ofrece mutex y variables de condición. Para bloquear un mutex Mueller no usa un simple test-and-set, porque el estándar pide protocolos que necesitan saber quién es el dueño del mutex, y ese dato tiene que quedar registrado de forma atómica junto con el bloqueo. Usa el `ldstub` de SPARC dentro de una secuencia atómica reinicializable de siete instrucciones: si una señal interrumpe la secuencia, el manejador la reinicia. De paso comenta que una instrucción compare-and-swap habría hecho todo esto más simple.

La parte que más me interesó es la de **inversión de prioridades**. El escenario es el de la Figura 5 del paper: un hilo de baja prioridad P_1 toma un mutex; aparece P_3 , de alta prioridad, lo desplaza y trata de tomar el mismo mutex, así que se bloquea. Si en ese momento hay un hilo de prioridad media P_2 listo, P_2 corre, P_1 nunca avanza, nunca libera el mutex y P_3 queda esperando a un hilo que ni siquiera usa el recurso. Las prioridades de P_2 y P_3 quedan, en la práctica, invertidas.

El estándar contempla dos soluciones, que el paper compara en su Tabla 3:

- **Herencia de prioridad:** cuando P_3 se bloquea en el mutex, el dueño P_1 hereda la prioridad de P_3 hasta que lo libere. Se adapta a cambios dinámicos de prioridad, pero para restaurar la prioridad correcta al liberar hay que recorrer linealmente los mutex que el hilo aún tiene tomados.

- **Techo de prioridad** (implementado con *Stack Resource Policy*): cada mutex tiene un techo fijo, al menos igual a la mayor prioridad de los hilos que lo pueden usar, y quien lo toma sube a ese techo de inmediato. La prioridad anterior se guarda en una pila, por lo que restaurarla es un simple *pop*. Mueller observa que este protocolo suele necesitar menos cambios de contexto y mantiene el mutex tomado menos tiempo.

También señala que mezclar los dos protocolos en secciones críticas anidadas rompe la disciplina de pila del techo (su Tabla 4), y propone que el estándar lo prohíba.

2.7 Perverted scheduling

Una sección que no esperaba encontrar en un paper de rendimiento es la de depuración. Los errores de concurrencia en multiprocesadores son difíciles de reproducir porque dependen del momento exacto en que cada procesador ejecuta cada instrucción. Mueller propone usar su librería en un uniprocador con políticas de planificación deliberadamente “perversas” que fuerzan conmutaciones donde normalmente no las habría:

- **Mutex switch:** cada vez que un hilo toma un mutex con éxito, se lo manda al final de su cola.
- **Round-robin ordered switch:** cada vez que se sale del kernel de la librería, se fuerza un cambio de contexto.
- **Random switch:** al salir del kernel se decide al azar si conmutar y a qué hilo.

La ventaja frente al round-robin con temporizador es que son deterministas: con la misma semilla se obtiene el mismo entrelazado, así que un error que aparece se puede volver a provocar. Con esto encontraron errores en su runtime de Ada que no se veían con la política FIFO.

2.8 Resultados reportados

La Tabla 1 recoge algunas de las mediciones del paper (Tabla 2 del original, columna de su implementación en SPARC IPX), que son el núcleo del argumento.

Tabla 1: Algunas mediciones de Mueller en SPARC IPX con SunOS 4.1 (en μs).

Operación	Tiempo (μs)
Entrar y salir del kernel de la librería	0.4
Entrar y salir del kernel UNIX (<code>getpid</code>)	18
Mutex lock/unlock sin contención	1
Cambio de contexto entre hilos (<code>yield</code>)	37
Cambio de contexto entre procesos UNIX	123
Manejador de señal, señal interna	52
Manejador de señal, señal externa	250

Entrar al kernel de la librería es unas 45 veces más barato que entrar al de UNIX, y cambiar de hilo es más de tres veces más barato que cambiar de proceso. La excepción son las señales externas: como la librería depende de los manejadores de UNIX, que ya son lentos, y encima tiene que redistribuirlas entre hilos, ese caso es caro. El propio autor concluye que el manejo de señales por hilo es lo que más complica el diseño, y que ahí es donde un soporte del kernel sí haría diferencia.

3 De SPARC en 1993 a Linux en 2026

Antes de escribir código tuve que decidir qué partes del paper eran esenciales y cuáles eran producto de la máquina de la época. Buena parte del texto está atado a SPARC: las *register windows*, el trap `ST_FLUSH_WINDOWS` para volcarlas a la pila, la instrucción `ldstub`, el registro reservado para el identificador del hilo. Nada de eso existe en el x86-64 donde corrí mis pruebas (Linux bajo WSL).

Lo que sí se conserva es la estructura del problema: hay que guardar y restaurar contextos en espacio de usuario, hay que apropiarse del procesador con una señal de temporizador, y hay que proteger las estructuras internas de esas señales. La Tabla 2 resume cómo traduje cada pieza.

La traducción más directa fue la del cambio de contexto. `ucontext` es una interfaz que POSIX ya marcó como obsoleta, pero sigue disponible en glibc y hace exactamente lo que necesitaba: `makecontext` prepara un contexto nuevo con su propia pila y una función de entrada, y `swapcontext` guarda el actual y salta al otro. Por eso el Makefile compila con `-Wno-deprecated-declarations`.

La que menos se parece es la del mutex. Mueller necesitaba una instrucción atómica porque quería que tomar un mutex sin contención no pasara por el kernel de la librería (le costaba 1 μ s). Yo tomé el camino fácil: todo `mpt_mutex.lock` corre dentro del kernel de la librería, así que la señal de temporizador no puede interrumpirlo a medias. Es correcto en un uniprocador, pero es precisamente lo que el paper intentaba evitar por costo.

4 Diseño e implementación de mini-pthreads

4.1 Organización del código

El proyecto está en C (estándar `gnu11`) y tiene alrededor de mil líneas en total. Se organiza así:

- `src/mpt.h`: interfaz pública. Define el bloque de control del hilo, el mutex, la variable de condición y todas las funciones.

- `src/mpt.internal.h`: funciones internas que comparten los módulos, como la entrada y salida del kernel y la cola de listos.
- `src/thread.c`: el núcleo. Cola de listos, dispatcher, kernel flags, manejador universal, temporizador, creación, cesión, join y salida de hilos, y políticas de planificación (unas 440 líneas).
- `src/mutex.c` y `src/cond.c`: mutex con los tres protocolos y variables de condición.
- `tests/`: cuatro programas de prueba.
- `Makefile`: compila la librería y las pruebas en `bin/`.

La Figura 1 muestra cómo encajan las piezas. Es la misma idea de capas de la Figura 1 del paper, pero con los módulos reales del proyecto.

4.2 Bloque de control y estados

Cada hilo está representado por una estructura `mpt_thread_t` (Código 1). Guarda dos prioridades: `base_prio`, la que se le asignó al crearlo, y `cur_prio`, la efectiva en ese momento, que puede subir temporalmente por herencia o por techo. El campo `ctx` es el `ucontext_t` donde se guarda su contexto cuando no está corriendo, y `next` lo enlaza en la cola de listos o en la lista de espera de un mutex o variable de condición.

```

1  typedef enum { MPT_READY, MPT_RUNNING, MPT_BLOCKED,
2                MPT_TERMINATED } mpt_state_t;
3
4  typedef struct mpt_thread {
5      int id;
6      int base_prio; /* prioridad asignada */
7      int cur_prio; /* prioridad efectiva (herencia /
8                    techo) */
9      mpt_state_t state;
10     ucontext_t ctx; /* contexto guardado */
11     char *stack; /* pila propia de 64 KB */
12     void (*entry)(void *arg);
13     void *arg;
14     struct mpt_thread *next; /* enlace en cola de
15                               listos o de espera */
16     struct mpt_thread *join_waiter; /* quien hizo
17                                       join sobre este hilo */
18     int has_exited;
19     int cs_count;
20 } mpt_thread_t;

```

Código 1: Bloque de control del hilo (`src/mpt.h`).

Los estados son los cuatro del paper: listo, corriendo, bloqueado y terminado. La Figura 2 muestra qué funciones de la librería producen cada transición.

4.3 Cola de listos y selección del siguiente hilo

Aquí me aparté a propósito del paper. Mueller tiene una cola por nivel de prioridad; yo tengo una sola lista enlazada con cabeza y cola. Insertar es siempre al final (`mpt_rq_enqueue`) y elegir al siguiente consiste en recorrer la lista entera y quedarse con el de mayor `cur_prio` (`pick_next`). Como la comparación es estricta (`>`), entre hilos de igual prioridad gana el que

Tabla 2: Correspondencia entre los mecanismos del artículo y mi implementación.

En el artículo (SunOS 4, SPARC)	En mini-pthreads (Linux, x86-64)
Volcado de register windows con <code>ST_FLUSH_WINDOWS</code> y <code>restore</code>	<code>swapcontext</code> de <code>ucontext.h</code> , que guarda registros y máscara de señales del hilo saliente y carga los del entrante
Temporizador de time-slicing	<code>setitimer(ITIMER_REAL)</code> que genera <code>SIGALRM</code> periódica
Manejador universal de señales	<code>universal_signal_handler()</code> instalado con <code>sigaction</code>
Kernel flag y dispatcher flag	<code>g_kernel_flag</code> y <code>g_dispatcher_flag</code> , declarados <code>volatile sig_atomic_t</code>
Entrada y salida del kernel	<code>mpt__enter_kernel()</code> y <code>mpt__leave_kernel()</code>
Colas de listos por prioridad	Una sola lista enlazada, recorrida linealmente buscando la mayor prioridad
Bloqueo de mutex con <code>ldstub</code> en secuencia atómica reiniciable	No hace falta instrucción atómica: el bloqueo ocurre dentro del kernel de la librería, protegido por el kernel flag
Herencia y techo de prioridad	Campo <code>protocol</code> del mutex: <code>MPT_NONE</code> , <code>MPT_INHERIT</code> , <code>MPT_CEILING</code>
Perverted scheduling	<code>mpt_set_sched_policy()</code> con mutex switch, round-robin ordered y random switch con semilla
Fake call empujada sobre la pila del hilo	Contexto auxiliar por hilo con su propia pila, activado antes de reanudar al hilo (prueba 4)

está más cerca de la cabeza, lo que da orden FIFO dentro de cada nivel.

El recorrido es $O(n)$, lo cual no importa con los pocos hilos de las pruebas. Pero tiene una consecuencia útil que describo en la sección de hallazgos: como la prioridad se lee en el momento de elegir y no al insertar, cambiarle la prioridad a un hilo que ya está en la cola no obliga a reubicarlo.

Cuando la política es *random switch*, `pick_next` ignora las prioridades y saca un hilo al azar usando `rand_r` con una semilla que se fija con `mpt_set_random_seed`. Uso `rand_r` y no `rand` para que el estado del generador sea una variable de la librería que puedo reiniciar, y así repetir exactamente una ejecución.

4.4 Entrada y salida del kernel de la librería

Entrar al kernel es trivial: `mpt__enter_kernel()` pone `g_kernel_flag` en 1. Toda la lógica está en la salida (Código 2). Si no hay nada que despachar, se baja el flag y se vuelve. Si el dispatcher flag está encendido, se decide si el hilo actual debe seguir o ceder, se elige al siguiente y se hace el cambio de contexto. Después del cambio, ya ejecutándose en el hilo que recibió el control, se revisa si llegó una señal diferida mientras se estaba dentro del kernel; si llegó, se repite el ciclo. Ese reintento es el mismo que describe la Figura 2 del paper.

```

1      apply_perverted_force(); /* round-robin ordered
2      / random switch */
3
4      for (;;) {
5          if (!g_dispatcher_flag) { /* nada que hacer
6              */
7              g_kernel_flag = 0;
8              return;
9          }
10         g_dispatcher_flag = 0;
11
12         mpt_thread_t *self = g_current;
13         if (self && self->state == MPT_RUNNING) {
14             mpt_thread_t *best_ready =
15                 rq_peek_highest();
16             if (!best_ready || best_ready->cur_prio
17                 <= self->cur_prio) {
18                 if (!g_deferred_signal) { /* si go
19                     yo */
20                     g_kernel_flag = 0;
21                     return;
22                 }
23                 g_deferred_signal = 0;
24                 continue;
25             }
26             self->state = MPT_READY; /* hay alguien
27                 mejor: cedo */
28             mpt__rq_enqueue(self);
29         }
30
31         mpt_thread_t *next = pick_next();
32         if (next != g_current)
33             do_switch(g_current, next); /*
34                 swapcontext */
35         else if (next)
36             next->state = MPT_RUNNING;
37
38         /* ya en el hilo que recibió el control */
39         if (!g_deferred_signal) {
40             g_kernel_flag = 0;
41             return;
42         }
43         g_deferred_signal = 0; /* llegó una señal:
44             reintentar */
45     }

```

Código 2: Núcleo de `mpt__leave_kernel()` con el monitor activado (`src/thread.c`, recortado).

Un punto que me gustó de este diseño es que el kernel flag no se baja antes del `swapcontext`, sino después, y lo baja el hilo que recibe el control. Mientras se está cambiando de contexto el flag sigue en 1, así que una

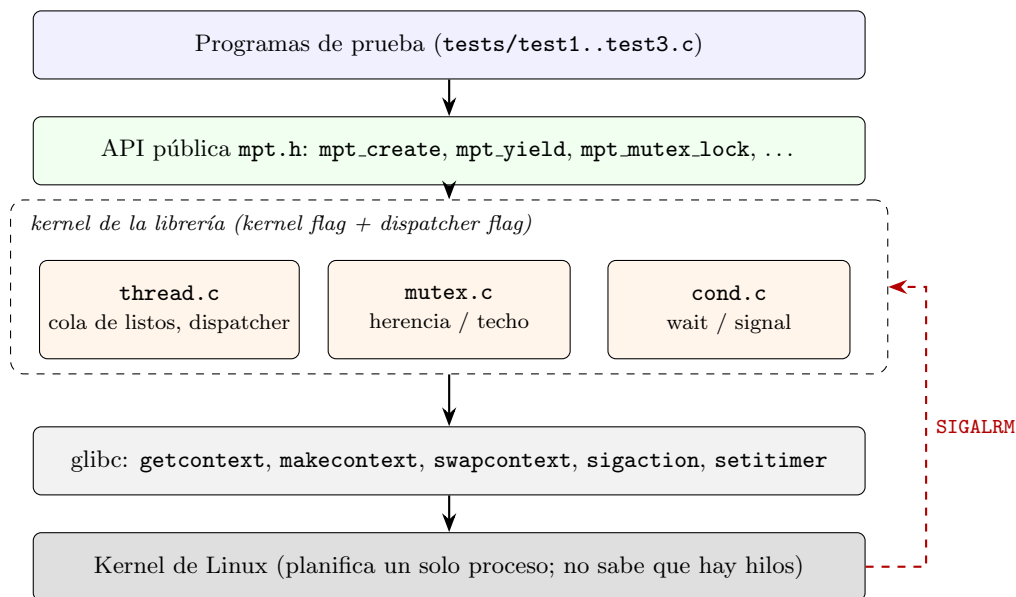


Figura 1: Capas de mini-pthreads. El kernel de Linux solo ve un proceso; los hilos existen únicamente dentro de la librería. La señal del temporizador sube desde el kernel y entra por el manejador universal.

señal que llegue justo en ese momento se difiere en lugar de iniciar un segundo despacho encima del primero.

La librería también permite apagar el monitor con `mpt_set_monitor_enabled(0)`. En ese modo la salida del kernel despacha directamente sin reintentos, y el manejador universal nunca difiere. Lo dejé como interruptor para poder comparar ambos comportamientos; además, en `mpt_rq_enqueue` hay un bucle vacío que solo se ejecuta con el monitor apagado y que ensancha la ventana entre leer la cola y enlazar el nodo nuevo, para que la corrupción de la cola sea observable en una ejecución corta y no dependa de que la señal caiga en un par de instrucciones.

4.5 Manejador universal y apropiación

La apropiación se activa con `mpt_enable_preemption(periodo)`, que instala el manejador con `sigaction` y programa `setitimer` para que envíe `SIGALRM` cada cierto número de microsegundos. El manejador (Código 3) sigue las dos ramas del paper: si llega dentro del kernel, solo anota la señal y enciende el dispatcher flag; si llega fuera, entra al kernel, devuelve al hilo actual al final de la cola (fin de su quantum) y sale del kernel, lo que dispara el despacho.

```

1  static void universal_signal_handler(int signo) {
2      (void)signo;
3      g_stat_signals++;
4
5      if (g_monitor_enabled && g_kernel_flag) {
6          /* dentro del kernel: registrar y diferir */
7          g_stat_deferred++;
8          g_deferred_signal = 1;
9          g_dispatcher_flag = 1;
10         return;
11     }
12
13     /* fuera del kernel: fin de quantum del hilo
14        actual */
15     mpt_enter_kernel();
16     if (g_current && g_current->state ==
17         MPT_RUNNING) {
18         g_current->state = MPT_READY;
19         mpt_rq_enqueue(g_current);
20     }
21     g_dispatcher_flag = 1;
22     mpt_leave_kernel();
23 }
  
```

Código 3: Manejador universal de señales (src/thread.c).

La regla de dirección de señales quedó simplificada. Como la única señal que maneja la librería es la del temporizador, siempre se dirige al hilo que estaba corriendo, que es el caso 3 del orden de precedencia del paper (el hilo que armó el temporizador) aplicado al time-slicing.

Las variables que comparten el manejador y el resto del código están declaradas `volatile sig_atomic_t`. Es lo mínimo que garantiza C para que una escritura hecha dentro de un manejador de señal sea vista correctamente por el código interrumpido.

4.6 Creación, cesión, join y salida

`mpt_create` reserva el bloque de control con `calloc` y una pila de 64 KB con `malloc`, prepara el contexto con

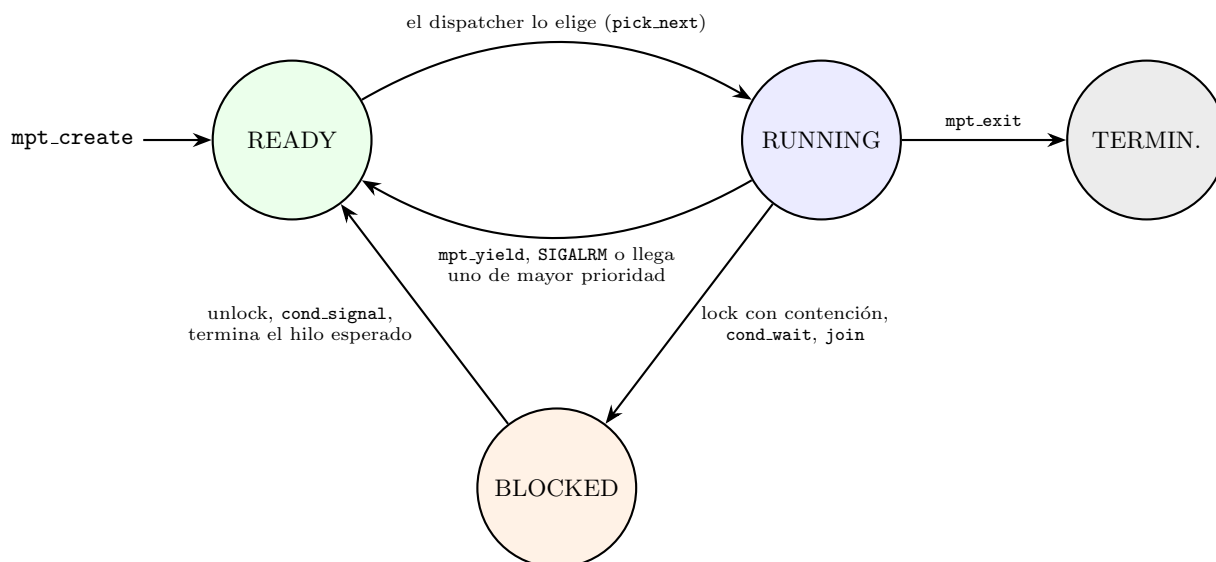


Figura 2: Estados de un hilo en mini-pthreads y las operaciones que los cambian.

`getcontext` y `makecontext` apuntando a una función `trampoline`, y encola el hilo dentro del kernel con el dispatcher flag encendido. Así, si el hilo nuevo tiene más prioridad que el que lo creó, el cambio de contexto ocurre al salir de `mpt_create`, como exige la planificación por prioridades.

La función `trampoline` llama a la función de entrada del usuario y, cuando esta retorna, llama a `mpt_exit`. Esto evita depender de `uc.link`: el hilo siempre termina pasando por la librería, que lo marca como terminado, despierta a quien estuviera esperando en `mpt_join` y despacha al siguiente. Cuando ya no quedan hilos listos, `pick_next` devuelve `NULL` y `do_switch` regresa al contexto `g_return_ctx`, que es el que guardó `mpt_run` en el hilo principal del programa. Así es como `mpt_run` retorna cuando todos los hilos terminaron.

4.7 Mutex y protocolos de prioridad

El mutex (Código 4) guarda su dueño, el protocolo, el techo, la prioridad que tenía el dueño antes de subir al techo y la lista de hilos bloqueados en él.

```

1 void mpt_mutex_lock(mpt_mutex_t *m) {
2     mpt__enter_kernel();
3     mpt_thread_t *self = g_current;
4
5     if (m->owner == NULL) { /* libre */
6         m->owner = self;
7         if (m->protocol == MPT_CEILING && m->
8             ceiling > self->cur_prio) {
9             m->saved_prio = self->cur_prio; /*
10                subir al techo */
11             self->cur_prio = m->ceiling;
12         }
13         if (mpt__sched_is_mutex_switch()) { /*
14             perverted: mutex switch */
15             self->state = MPT_READY;
16             mpt__rq_enqueue(self);
17             mpt__force_dispatch();
18         }
19     } else { /* ocupado */
20         if (m->protocol == MPT_INHERIT && self->
21             cur_prio > m->owner->cur_prio)
22             boost_owner_priority(m->owner, self->
23                 cur_prio); /* herencia */
24
25         self->state = MPT_BLOCKED;
26         mpt__wlist_add(&m->wait_head, self);
27         mpt__force_dispatch();
28     }
29     mpt__leave_kernel();
30 }
  
```

Código 4: Bloqueo de mutex con los tres protocolos (`src/mutex.c`).

Con **techo**, el hilo sube de prioridad al tomar el mutex, sin esperar a que haya contención, y guarda su prioridad anterior en `saved_prio`. Con **herencia**, no pasa nada al tomar el mutex; la subida ocurre cuando otro hilo de mayor prioridad intenta tomarlo y encuentra dueño. Al liberar, el techo restaura `saved_prio` y la herencia vuelve a `base_prio`. Luego se entrega el mutex directamente al hilo en espera de mayor prioridad: el mutex no queda libre ni por un instante, lo cual evita que un tercer hilo se lo gane al que despertó.

La política *mutex switch* también vive aquí: si está activa, tomar un mutex con éxito manda al hilo al final de la cola y fuerza un despacho, tal como la describe Mueller.

4.8 Variables de condición

`mpt_cond_wait` hace, dentro del kernel, las tres cosas que tienen que ocurrir juntas: marca al hilo como bloqueado, lo agrega a la lista de la variable de condición y libera el mutex. Si esto no fuera atómico, podría colarse un `signal` entre la liberación y el bloqueo y el hilo se dormiría para siempre. Al despertar, vuelve a tomar el mutex con `mpt_mutex_lock`. `mpt_cond_signal` despierta al hilo en espera de mayor prioridad, que es lo que pide el paper, y `broadcast` los despierta a todos.

4.9 Fake calls

La cuarta prueba es un programa autocontenido que no usa `src/`. Lo hice así porque meter las fake calls dentro de la librería completa mezclaba demasiadas cosas a la vez, y quería poder mostrar el mecanismo aislado.

Hay dos hilos cooperativos, cada uno con un manejador de usuario registrado y un campo `pending_signal`. Un temporizador real envía `SIGALRM` cada 150 ms, y el manejador universal no ejecuta nada del usuario: solo elige un hilo destino (por turnos) y le marca la señal como pendiente. La parte interesante está en el dispatcher (Código 5).

```

1 static void fake_call_wrapper(void) {
2     thread_t *t = wrapping;
3     t->handler(t->id, t->pending_signal); /*
4         handler de usuario */
5     t->pending_signal = 0;
6     setcontext(&t->ctx); /* volver al punto exacto
7         */
8 }
9
10 static void dispatch(int target) {
11     current = target;
12     thread_t *t = &threads[target];
13
14     if (t->pending_signal) { /* hay una señal
15         diferida */
16         getcontext(&t->wrap_ctx);
17         t->wrap_ctx.uc_stack.ss_sp = t->wrap_stack;
18         t->wrap_ctx.uc_stack.ss_size =
19             WRAP_STACK_SIZE;
20         wrapping = t;
21         makecontext(&t->wrap_ctx, fake_call_wrapper,
22             0);
23         swapcontext(&main_ctx, &t->wrap_ctx); /*
24             primero el wrapper */
25     } else {
26         swapcontext(&main_ctx, &t->ctx); /*
27             reanudar normal */
28     }
29 }

```

Código 5: Despacho con fake call
(tests/test4_fakecall.c).

Si el hilo que se va a despachar tiene una señal pendiente, en vez de saltar a su contexto se salta a un contexto intermedio que ejecuta el wrapper. El wrapper llama al manejador de usuario y termina con `setcontext` sobre el contexto guardado del hilo, que lo devuelve exactamente al punto donde cedió. Visto desde el hilo, el manejador se ejecutó como si lo hubiera llamado él mismo justo antes de continuar.

La diferencia con el paper es dónde vive el marco del wrapper. Mueller lo empuja sobre la misma pila del

hilo, modificando su contador de programa y su puntero de pila. Con `ucontext` no tengo una forma limpia de hacer eso sin tocar a mano los registros guardados, que dependen de la arquitectura, así que le di al wrapper una pila auxiliar propia de 32 KB por hilo. El comportamiento que se observa desde fuera es el mismo: la señal se difiere, el manejador corre en el turno del hilo receptor y el hilo continúa donde estaba.

5 Pruebas y resultados

5.1 Entorno

Compilé y ejecuté todo en Ubuntu sobre WSL, en una laptop con Windows, con `cc` (GCC) y las opciones del Makefile:

```

CFLAGS = -std=gnu11 -Wall -Wextra -g -O0 -
D_XOPEN_SOURCE=700 \
-Wno-deprecated-declarations
make # genera bin/test1_monitor ... bin/
test4_fakecall

```

Código 6: Compilación.

Uso `-O0` a propósito. Con optimizaciones el compilador puede reordenar o eliminar accesos a variables que, desde su punto de vista, nadie más modifica, y en este código esas variables las modifica un manejador de señales o las observa otro contexto después de un `swapcontext`.

Las pruebas usan `assert` para validar el resultado esperado. Si una aserción falla, el programa aborta; si todo está bien, imprime OK.

5.2 Prueba 1: productor y consumidor sobre el kernel de la librería

La primera prueba ejercita el camino completo por el kernel de la librería con sincronización real. Crea un consumidor y un productor de igual prioridad que comparten un mutex (con protocolo de herencia) y una variable de condición. El consumidor toma el mutex y, mientras no haya un elemento listo, espera en la variable de condición. El productor toma el mutex, marca el elemento como listo, señala la condición y libera.

```

root@LAPTOP-03D5EE88:/mnt/c/Users/Dario/Desktop/P1_S0# ./bin/test1_monitor
test1_monitor: OK

```

Figura 3: Ejecución de `test1_monitor`.

Siguiendo el código, el orden de ejecución es el siguiente. Al crear al consumidor no hay hilo corriendo (estamos en `main`), así que `mpt_leave_kernel` retorna sin despachar; lo mismo con el productor. `mpt_run` elige al consumidor, que es el primero en la cola. Este toma el mutex libre, ve que no hay elemento y entra a `mpt_cond_wait`: dentro del kernel se bloquea, se agrega a la lista de la condición y libera el mutex, y al salir del kernel el dispatcher elige al productor. El productor toma el mutex, marca el elemento, y

`mpt_cond_signal` devuelve al consumidor a la cola de listos. Como tienen la misma prioridad, el productor sigue, libera el mutex y termina. El consumidor retoma dentro de `mpt_cond_wait`, vuelve a tomar el mutex, comprueba el predicado en el `while`, lo encuentra verdadero y termina.

Que la prueba pase confirma varias cosas a la vez: que la liberación del mutex y el bloqueo en la condición son atómicos respecto del resto, que el `while` alrededor del `wait` funciona (el paper insiste en reevaluar el predicado al despertar), y que `mpt_run` retorna al `main` cuando todos los hilos terminaron.

5.3 Prueba 2: el escenario de inversión de prioridades

Esta prueba reproduce la Figura 5(b) del paper. Solo se crea desde `main` el hilo P_1 con prioridad 1. P_1 toma el mutex (con protocolo de herencia) y, ya dentro de la sección crítica, crea a P_3 con prioridad 3 y a P_2 con prioridad 2. Luego cede con `mpt_yield` y libera el mutex. P_3 intenta tomar el mismo mutex y P_2 solo cede el procesador cinco veces, sin tocar el recurso.

```
root@LAPTOP-03D5EE88:/mnt/c/Users/Dario/Desktop/P1_S0# ./bin/test2_inversion
test2_inversion: OK
```

Figura 4: Ejecución de `test2_inversion`.

La secuencia, siguiendo el código, es la que muestra la Figura 5:

1. P_1 toma el mutex libre y crea a P_3 . Al salir de `mpt_create`, el dispatcher ve que hay un hilo listo con prioridad 3 mayor que la de P_1 , así que P_1 vuelve a la cola y corre P_3 .
2. P_3 intenta tomar el mutex, encuentra como dueño a P_1 y, como el protocolo es herencia, le sube la prioridad a 3. P_3 se bloquea.
3. El dispatcher elige al de mayor prioridad en la cola, que ahora es P_1 con prioridad heredada 3. P_1 crea a P_2 , pero P_2 tiene prioridad 2 y no lo desplaza.
4. P_1 cede con `mpt_yield`. En la cola están P_2 (2) y P_1 (3), así que vuelve a correr P_1 .
5. P_1 libera el mutex: vuelve a su prioridad base 1, el mutex pasa directamente a P_3 y P_3 entra a la cola. Al salir del kernel, P_3 desplaza a P_1 , completa su sección crítica y termina.
6. Recién entonces corre P_2 y, al final, P_1 termina.

Lo importante es el punto 6: P_2 no llega a ejecutar ni una vez mientras P_3 está bloqueado. Sin herencia, en el paso 3 P_1 seguiría con prioridad 1, al crear a P_2 este lo desplazaría y haría sus cinco `yield` antes de que P_1 pudiera liberar el mutex; ese es exactamente el caso (a) de la Figura 5. La aserción final solo comprueba que

P_3 llegó a ejecutar su sección crítica, pero el orden que se deduce del código es el que ilustra el paper para el caso (b).

5.4 Prueba 3: despacho de varios hilos

La tercera prueba crea cinco hilos de igual prioridad que solo incrementan un contador compartido, llama a `mpt_run` y verifica que el contador llegó a 5.

```
root@LAPTOP-03D5EE88:/mnt/c/Users/Dario/Desktop/P1_S0# ./bin/test3_pervverted
test3_pervverted: OK
```

Figura 6: Ejecución de `test3_pervverted`.

Es la prueba más simple, pero valida el ciclo de vida completo con varios hilos: que la cola de listos entrega a cada hilo exactamente una vez, que `trampoline` llama a `mpt_exit` al terminar la función del usuario, que cada salida despacha al siguiente y que, al vaciarse la cola, el control vuelve a `main`. Es la base sobre la que se montan las políticas perversas: `mpt_set_sched_policy` cambia la forma en que `pick_next` y `mpt__leave_kernel` eligen y fuerzan conmutaciones, pero todo el resto del recorrido es el que ejercita esta prueba. Tal como está, la prueba corre con la política FIFO por defecto; en la sección 6 comento cómo la extendería.

5.5 Prueba 4: fake calls

Esta es la prueba con la salida más rica, y la que mejor muestra la idea del paper.

Los dos hilos se alternan dando cinco pasos cada uno. El dispatcher duerme 40 ms entre despacho y despacho, y la señal llega cada 150 ms, así que cae aproximadamente cada tres o cuatro despachos. En la ejecución de la Figura 7 llegaron tres señales, y cada una ilustra un caso distinto:

- **Señal #1.** Llega cuando el último en correr fue el hilo 1 y va dirigida al hilo 0, que no está corriendo. No se ejecuta nada en ese momento; solo se marca como pendiente. En el siguiente despacho del hilo 0, antes de su paso 3, aparece la fake call: se ejecuta su manejador (con su prioridad, 1) y luego el hilo retoma exactamente donde había cedido, e imprime **paso 3/5**.
- **Señal #2.** Esta vez va dirigida al propio hilo 1, que era el último en correr. Aun así no se ejecuta en el acto: se atiende cuando el hilo 1 vuelve a ser despachado, después de que el hilo 0 da su paso 5. Esto muestra que el mecanismo no depende de quién estaba corriendo: el manejador siempre se ejecuta en el turno del hilo receptor.
- **Señal #3.** Llega cuando ambos hilos ya terminaron. Queda marcada como pendiente para el hilo 0 pero nunca se atiende, porque un hilo terminado

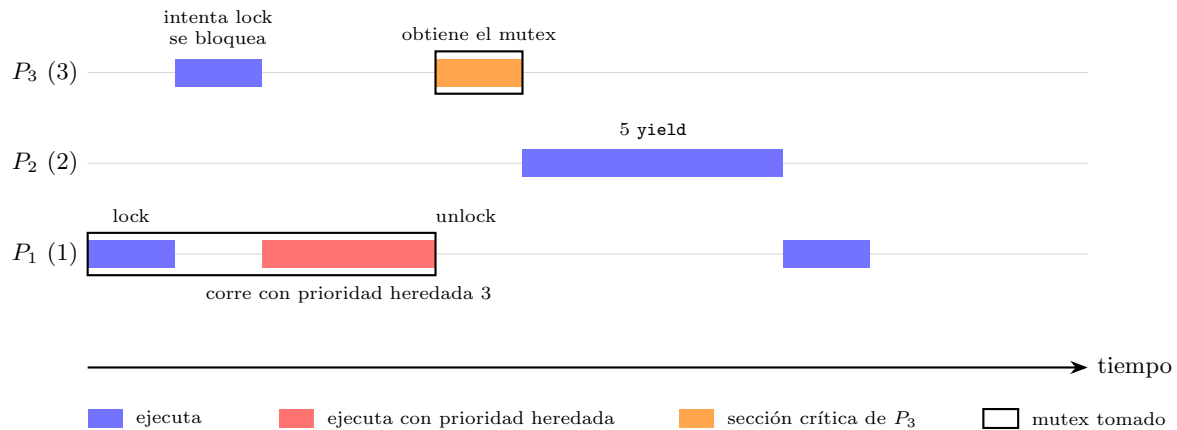


Figura 5: Orden de ejecución de la prueba 2 con herencia de prioridad (esquemático; los anchos no representan tiempos medidos). P_2 no corre mientras P_3 espera el mutex.

```

root@LAPTOP-03D5EE88:/mnt/c/Users/Dario/Desktop/P1_S0# ./bin/test4_fakecall
Demo 4: fake calls (Figura 3 del paper de Mueller)
2 hilos cooperativos; SIGALRM real cada 150 ms, dirigida por turnos a un hilo distinto del que este corriendo

hilo 0: paso 1/5 (prioridad 1)
hilo 1: paso 1/5 (prioridad 2)
hilo 0: paso 2/5 (prioridad 1)
hilo 1: paso 2/5 (prioridad 2)
[SIGALRM #1] llega mientras corre hilo 1 -> dirigida al hilo 0 (que NO esta corriendo): se DIFIERE
-> [fake call] hilo 0 ejecuta AHORA su handler para la señal 14 (prioridad 1), justo antes de reanudar su codigo
(handler de usuario) hilo 0 atiende la señal 14: actualiza su estado interno y sigue
<- [fake call] hilo 0 retorna exactamente al punto donde habia quedado
hilo 0: paso 3/5 (prioridad 1)
hilo 1: paso 3/5 (prioridad 2)
hilo 0: paso 4/5 (prioridad 1)
hilo 1: paso 4/5 (prioridad 2)
[SIGALRM #2] llega mientras corre el propio hilo 1 -> se difiere igual, se atendera en su proximo despacho
hilo 0: paso 5/5 (prioridad 1)
-> [fake call] hilo 1 ejecuta AHORA su handler para la señal 14 (prioridad 2), justo antes de reanudar su codigo
(handler de usuario) hilo 1 atiende la señal 14: actualiza su estado interno y sigue
<- [fake call] hilo 1 retorna exactamente al punto donde habia quedado
hilo 1: paso 5/5 (prioridad 2)
hilo 0: termino
hilo 1: termino
[SIGALRM #3] llega mientras corre hilo 1 -> dirigida al hilo 0 (que NO esta corriendo): se DIFIERE

test4_fakecall: OK

```

Figura 7: Ejecución de `test4.fakecall`.

ya no se despacha. Es coherente con el paper, que establece que un hilo terminado no puede volver a planificarse.

En los tres casos el orden de los pasos de cada hilo se mantiene intacto (1, 2, 3, 4, 5), lo que confirma que el wrapper devuelve el control al punto correcto y que la fake call es transparente para el código del hilo.

5.6 Resumen

La Tabla 3 junta las cuatro pruebas, lo que verifica cada una y la sección del artículo con la que se relaciona. Las cuatro terminan con OK.

6 Hallazgos y discusión

Más allá de que las pruebas pasen, al revisar con cuidado el código y compararlo con el paper encontré varias cosas que vale la pena anotar. Algunas son decisiones que salieron mejor de lo esperado y otras son diferencias o debilidades que prefiero dejar escritas.

La cola única simplifica la herencia. Cuando P_3 le sube la prioridad a P_1 , P_1 está en la cola de listos (había sido desplazado al crear a P_3). Con una cola por prioridad, como en el paper, habría que sacarlo de la cola de nivel 1 y meterlo en la de nivel 3; si no se hace, el dispatcher lo buscaría en el nivel equivocado. En mi implementación `boost.owner.priority` solo cambia `cur_prio`, y funciona porque `pick_next` recorre toda la lista y lee la prioridad en ese momento. Es un ejemplo de cómo una estructura más lenta ($O(n)$)

Tabla 3: Resumen de las pruebas.

Prueba	Qué verifica	Sección del paper	Resultado
<code>test1_monitor</code>	Mutex y variable de condición a través del kernel de la librería; bloqueo y liberación atómicos	Pthreads Kernel, Synchronization	OK
<code>test2_inversion</code>	P_3 obtiene el mutex sin que P_2 corra antes, gracias a la herencia	Priority Inversion, Fig. 5(b)	OK
<code>test3_perverted</code>	Ciclo de vida completo de varios hilos y retorno a <code>main</code>	Thread States, Dispatcher	OK
<code>test4_fakecall</code>	Señales diferidas y manejador ejecutado en el turno del hilo receptor	Fake Calls, Fig. 3	OK

puede eliminar una clase entera de errores. A cambio, el costo de elegir al siguiente hilo crece con la cantidad de hilos listos, algo que Mueller no se podía permitir.

El techo de prioridad y el yield dentro de la sección crítica. Revisando qué pasaría si la prueba 2 usara techo en lugar de herencia, noté un detalle de la política FIFO. Con techo 3, P_1 sube a prioridad 3 apenas toma el mutex, así que al crear a P_3 (también 3) no es desplazado, porque el dispatcher solo cambia de hilo si el otro tiene prioridad estrictamente mayor. Pero cuando P_1 hace `mpt_yield`, va al final de la cola, detrás de P_3 , que tiene la misma prioridad. P_3 corre, intenta tomar el mutex y se bloquea. En el caso ideal de la Figura 5(c), P_3 nunca llega a intentar el lock mientras P_1 lo tiene. La diferencia la causa el `yield` explícito dentro de la sección crítica, que en un programa real no tendría mucho sentido; aun así, muestra que el techo solo evita la contención si el dueño no cede voluntariamente, y conecta con la observación del paper sobre si un hilo que vuelve de un cambio de prioridad debe ir a la cabeza o a la cola de su nivel.

Restauración de prioridad simplificada. Al liberar un mutex con herencia, el hilo vuelve directamente a `base_prio`. El paper, en cambio, calcula el máximo entre su prioridad propia y la de los hilos que esperan en otros mutex que todavía tiene tomados. Con un solo mutex, como en la prueba 2, da igual. Con mutex anidados podría reaparecer la inversión: si P_1 tiene dos mutex con esperas y libera uno, perdería la prioridad heredada por el otro. Lo mismo pasa con el techo: `saved_prio` es un único valor por mutex y no una pila por hilo, así que funciona mientras los mutex se liberen en orden inverso al que se tomaron. Es justamente la condición que Mueller pide que el estándar exija.

Desenmascarar SIGALRM al conmutar. En `do_switch`, antes del `swapcontext`, se quita SIGALRM de la máscara guardada del hilo entrante. Lo hice para que un hilo que fue interrumpido desde el manejador no se quede con la señal bloqueada para siempre. Pero el paper recomienda exactamente lo contrario: enmascarar las señales antes de conmutar hacia un hilo interrumpido, y dejar que se desenmascaren solas al retornar del manejador. Con mi versión existe una

ventana pequeña, entre que el hilo retoma dentro del manejador y que este retorna, en la que puede llegar otra SIGALRM y apilarse otro manejador. En la práctica no lo he visto fallar porque esa ventana es de muy pocas instrucciones y el periodo del temporizador es mucho mayor, pero es una desviación real del objetivo de no crecer la pila sin límite, y la corregiría distinguiendo si el hilo entrante fue interrumpido o cedió voluntariamente.

Funciones no reentrantes dentro del manejador.

En la prueba 4 el manejador universal llama a `printf` para narrar lo que pasa. `printf` no está en la lista de funciones seguras para señales (*async-signal-safe*): si la señal llegara mientras un hilo está en medio de otro `printf`, el búfer de `stdout` podría quedar inconsistente. Lo dejé así porque la prueba es didáctica y el temporizador es lento, pero en la librería real el manejador solo toca variables `sig_atomic_t`, que es lo correcto. Es el mismo problema de fondo que Mueller menciona al final sobre hacer reentrantes las librerías de C.

Memoria de los hilos terminados. El diagrama del dispatcher del paper incluye un paso para liberar la memoria del hilo que termina. En mi implementación la pila y el bloque de control de un hilo terminado no se liberan. No se puede hacer un `free` de la pila desde el propio hilo mientras todavía se ejecuta sobre ella, así que habría que hacerlo después del cambio de contexto, desde el hilo siguiente. Con pocos hilos no importa, pero en un programa que cree hilos continuamente sería una fuga. Mueller además comenta que el 70% del tiempo de creación se va en reservar pila y bloque de control, y propone un pool; eso resolvería las dos cosas a la vez.

Cobertura de las pruebas. La librería implementa la apropiación por temporizador, el modo sin monitor y las tres políticas perversas, pero las pruebas 1 a 3 corren con planificación cooperativa y política FIFO. La prueba 4 sí usa un temporizador real, pero sobre su propio despachador. Las siguientes extensiones naturales serían: una prueba que active `mpt_enable_preemption` y compare el conteo de señales diferidas con el monitor activado y desactivado, una versión de la prueba 3 con

una carrera de tipo verificar-y-luego-actuar que pase con FIFO y falle con MPT_SCHED_MUTEX_SWITCH o con ciertas semillas de MPT_SCHED_RANDOM, y una prueba 2 que ejecute los tres protocolos y compare el orden de ejecución y la cantidad de cambios de contexto con `mpt_context_switches()`. La infraestructura para las tres ya existe en la librería.

No hacen falta instrucciones atómicas en un uniprosesor. Quizá lo más interesante que saqué de comparar ambas implementaciones es que toda la complejidad del bloqueo de mutex del paper (`ldstub`, secuencias reiniciables, registro del dueño) existe para que el caso sin contención no pase por el kernel de la librería. Si uno acepta pagar ese costo, el kernel flag basta para toda la sincronización en un solo procesador, porque la única fuente de concurrencia es la señal y la señal se difiere. En un multiprosesor esto ya no sería cierto: dos procesadores pueden estar dentro del kernel a la vez y el flag no protegería nada. Por eso Mueller dice que para multiprosesores habría que reemplazar el monitor monolítico por candados finos.

7 Limitaciones

Además de lo mencionado en la sección anterior, conviene dejar claros los límites de este trabajo:

- No medí tiempos. El argumento central del paper es de rendimiento, y mi implementación no reproduce la Tabla 2. Una medición honesta requeriría comparar, por ejemplo, el costo de `mpt__enter_kernel` y `mpt__leave_kernel` contra `getpid`, y `mpt_yield` contra un cambio entre procesos, en la misma máquina. Además `swapcontext` en glibc hace una llamada al sistema para guardar y restaurar la máscara de señales, lo que de entrada lo hace más caro que el cambio de contexto del paper.
- Solo se maneja SIGALRM. No hay dirección de señales síncronas, máscaras por hilo, `sigwait` ni cancelación.
- Todo corre en un solo procesador, igual que en el paper.
- La fake call se muestra en un programa aparte y con una pila auxiliar, no integrada en la librería ni empujada sobre la pila del hilo.
- `ucontext` está marcado como obsoleto en POSIX. Funciona en glibc, pero no es una base con futuro para una librería real.

8 Conclusiones

El artículo de Mueller sostiene que una implementación de Pthreads hecha completamente como librería de

usuario es factible y competitiva. Después de implementar una versión reducida en Linux, la parte de factibilidad me parece clarísima: con `ucontext`, `sigaction` y `setitimer` se puede tener hilos con prioridades, apropiación, mutex con protocolos contra la inversión y señales diferidas, sin que el kernel del sistema operativo sepa que existen.

La idea que más me quedó es la del kernel de la librería. Dos variables bastan para convertir un manejador de señales, que puede interrumpir cualquier instrucción, en algo que solo actúa en puntos seguros. Todo lo demás se apoya en eso: el dispatcher puede asumir que las colas son consistentes, el mutex no necesita instrucciones atómicas en un uniprosesor y las señales se pueden redirigir a otros hilos sin correr el riesgo de actuar sobre estructuras a medio modificar.

También entendí mejor por qué Mueller termina diciendo que las señales son lo más difícil. En mi implementación la parte de señales es la más corta en líneas y, de lejos, la más delicada: es donde están la desviación sobre el enmascaramiento, el uso de funciones no reentrantes en la prueba 4 y la necesidad de compilar sin optimizaciones. Treinta años después, la mayoría de sistemas resolvieron esto llevando los hilos al kernel, que es la alternativa que el propio autor sugiere al final.

Las cuatro pruebas pasan y cada una corresponde a una sección concreta del paper. Los hallazgos de la sección 6 marcan lo que falta para que la librería sea más fiel: restaurar prioridades con una pila por hilo, enmascarar la señal al volver a un hilo interrumpido, liberar la memoria de los hilos terminados y ampliar las pruebas para usar la apropiación y las políticas perversas que ya están implementadas.

Referencias

- [1] F. Mueller. A Library Implementation of POSIX Threads under UNIX. En *Proceedings of the Winter 1993 USENIX Conference*, pp. 29–41, San Diego, CA, enero de 1993.
- [2] IEEE. *Threads Extension for Portable Operating Systems (Draft 6)*. P1003.4a/D6, febrero de 1992.
- [3] L. Sha, R. Rajkumar y J. P. Lehoczky. Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on Computers*, 39(9):1175–1185, 1990.
- [4] T. P. Baker. Stack-based scheduling of realtime processes. *Real-Time Systems*, 3(1):67–99, 1991.
- [5] The Linux man-pages project. `getcontext(3)`, `makecontext(3)`, `sigaction(2)`, `setitimer(2)`, `signal-safety(7)`.